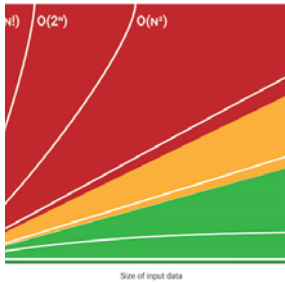


B 2.4.1 Big O



B 2.4.1 Describe the efficiency of specific algorithms by calculating their Big O notation to analyse their scalability.

This section provides a comprehensive guide to understanding and analysing algorithm efficiency, a key concept in computer science. We'll explore how to evaluate algorithms using Big O notation, calculate time and space complexities, and learn how to choose the right algorithm for different situations.

Last updated: 12 Oct 2025

Learning Objectives

- Students will be able to define and explain the concept of Big O notation.
- Students will be able to analyse the time and space complexity of algorithms using Big O notation.
- Students will be able to evaluate the efficiency of different algorithms based on their time and space complexity.
- Students will be able to select appropriate algorithms based on scalability and efficiency requirements.

IB ATL Skills

Thinking: Critical thinking, creative thinking, transfer

Research: Information literacy, media literacy

Communication: Communication, collaboration

Lesson Resources

Presentations

The presentation offers an introduction to the topic that can be used to give a structured explanation on the topic.

Java Presentation

[Link to presentation](#)

Python Presentation

[Link to presentation](#)

eBook

The eBook section contains all the information students require to find out more. It can be assigned as a reading task or even used in a lesson in presentation mode.

B 2.4.1 Evaluating Algorithm Efficiency with Big O Notation

Worksheets

There are separate worksheets for Java and Python that you can make available to students along with an answer key sheet.

B 2.4.1 Worksheet Java

B 2.4.1 Worksheet Python

B 2.4.1 Worksheet Answers Java

B 2.4.1 Worksheet Answers Python

Lesson Plan Java

Stage	Time	Activity & Teacher Focus	Content Covered (Slide #)
I. Introduction & Context	10 min	Engage & Review: Start with the concept of an algorithm as a "recipe" (Slide: What is an Algorithm?). Discuss Why Efficiency Matters using the "huge phone book" analogy to introduce the need for better methods than checking every item.	Slides 1-3
II. Time Complexity Fundamentals	15 min	Explain $O(1)$ & $O(n)$: Focus on the cause of the complexity. $O(1)$ is direct access (index access). $O(n)$ is linear traversal (single loop). Use the code snippets to solidify the structural difference.	Slides 4-7
III. Advanced Time Complexity	15 min	Explain $O(n^2)$ & $O(\log n)$: Contrast the two. $O(n^2)$ is the scalability killer (nested loops, comparing every item). $O(\log n)$ is the efficiency booster (divide and conquer, halving the search space). Relate $O(\log n)$ specifically to the Binary Search concept.	Slides 8-11
IV. Space Complexity Analysis	10 min	Explain Space Complexity: Define the difference between Time and	Slides 12-16

Stage	Time	Activity & Teacher Focus	Content Covered (Slide #)
		Space Complexity. Walk through the three space categories ($O(1)$, $O(n)$, $O(n^2)$) using the provided Java code snippets. Focus on auxiliary space—memory created by the algorithm itself (e.g., the new <code>int[n][n]</code> for $O(n^2)$).	
V. Application & Synthesis	10 min	Choosing the Right Algorithm: Review the slide on decision-making: Scalability vs. Efficiency Requirements. Lead a quick class vote: "Which Big O would you choose for an app that serves 1 billion users?" (Answer: $O(1)$ or $O(\log n)$).	Slide 17
VI. Conclusion & Activity	10 min	Activity/Worksheet Instructions: Distribute the Java Worksheet. Instruct students to analyse the snippets, identify the complexity, and state the reasoning. Review the Conclusion slide.	Slides 18-19

Lesson Plan Python

Stage	Time	Activity & Teacher Focus	Content Covered (Slide #)
I. Introduction & Context	10 min	Engage & Review: Start with the concept of an algorithm as a "recipe." Discuss Why Efficiency Matters using the "huge phone book" analogy. Introduce Big O as the tool for measuring scalability.	Slides 1-3
II. Time Complexity Fundamentals	15 min	Explain $O(1)$ & $O(n)$: Focus on the cause of the complexity. $O(1)$ is direct access (list index or dictionary key access). $O(n)$ is linear traversal (single loop). Use the Python code snippets to solidify the structural difference.	Slides 4-7
III. Advanced Time Complexity	15 min	Explain $O(n^2)$ & $O(\log n)$: Contrast the two. $O(n^2)$ is the scalability killer (nested loops). $O(\log n)$ is the efficiency booster	Slides 8-11

Stage	Time	Activity & Teacher Focus	Content Covered (Slide #)
		(halving the search space). Highlight the Python-specific use of integer division (//) in the $O(\log n)$ binary search.	
IV. Space Complexity Analysis	10 min	Explain Space Complexity: Define the difference between Time and Space Complexity. Walk through the three space categories ($O(1)$, $O(n)$, $O(n^2)$) using the provided Python code snippets. Focus on the use of <code>set()</code> for $O(n)$ space and list comprehension for $O(n^2)$ space.	Slides 12-16
V. Application & Synthesis	10 min	Choosing the Right Algorithm: Review the slide on decision-making: Scalability vs. Efficiency Requirements. Lead a quick class vote: "Which Big O is generally too slow for an app that serves 1 million users?" (Answer: $O(n^2)$).	Slide 17
VI. Conclusion & Activity	10 min	Activity/Worksheet Instructions: Distribute the Python Worksheet. Instruct students to analyse the snippets, identify the complexity, and state the reasoning. Review the Conclusion slide.	Slides 18-19

Student Challenges and Alleviation Strategies

Conceptual Challenges

Challenge	Description	Alleviation Strategy
Understanding Asymptotics	Students often confuse Big O notation (which describes growth rate for large n) with actual runtime (seconds). They may argue an $O(n^2)$ is faster than $O(n)$ because it runs faster for $n=5$.	Use the Graph (Slide 4): Consistently refer students to the Big O graph. Emphasise the phrase "as the input size approaches infinity." Show a simple example where $100n$ is greater than n^2 for small n , but n^2 quickly overtakes it.
Dominant Term (Simplification)	Students struggle with dropping constants ($5n$ simplifies to $O(n)$) and lower-order terms (n^2+n simplifies to $O(n^2)$).	The "Infinity Test": Ask students: "If n is 1 billion, which term contributes most to the final number?" (Answer: n^2). Explain that compared to n^2 , n and constants are negligible at large scale.
Space vs. Time	Students often mistakenly apply time analysis rules to space complexity, or forget to account for auxiliary space.	Explicit T-Diagrams: When analysing code, use a two-column chart: Time Counter (for steps) and Space Counter (for variables/data structures created inside the function). Highlight variables like the temp variable in $O(1)$ swap vs. a HashSet in $O(n)$ space.

Algorithmic Challenges

Challenge	Description	Alleviation Strategy
Identifying $O(\log n)$ Structure	Students may not recognise the pattern that causes logarithmic complexity, often confusing it with $O(n)$ loops.	The Halving Rule: Teach students that $O(\log n)$ is generated only when the problem size is reduced by a fixed fraction (usually $1/2$) in every iteration. Contrast the binary search ($\text{low} = \text{mid} + 1$) with a linear search loop that checks every item.
Identifying $O(n^2)$ Structure	Students sometimes miss or miscount nested loops that contribute to $O(n^2)$ (e.g., confusing for i in $\text{range}(10)$ inside a loop with a loop dependent on n).	Nesting Rule: Reinforce the rule: If a loop runs $O(n)$ times and contains another loop that runs $O(n)$ times, the result is $O(n^2)$. Have students physically draw a multiplication grid to show why $5 \times 5 = 25$ steps.

being blocked and legal action being taken against you.

 **Report a problem**